



Database Management Systems

Aggregate Functions

Anchor:

Dr. Nagasundari S

Department of Computer Science and Engineering

Database Management Systems

Unit 2: Advanced SQL in Database Design

Slides adapted from Author Slides of “Database System Concepts”, Silberschatz, H Korth and S Sudarshan, McGrawHill, 7th Edition, 2019. And Author Slides of Fundamentals of Database Systems”, Ramez Elamsri, Shamkant B Navathe, Pearson, 7th Edition, 2017.

Dr. Nagasundari S

Department of Computer Science and Engineering

- Aggregate functions are used to summarize information from multiple tuples into a single-tuple summary.
- Grouping is used to create subgroups of tuples before summarization.

Built-in aggregate functions:

- **COUNT** - returns the number of tuples or values specified in a query
- **SUM** – returns the sum of a set (or multiset) of numeric values
- **MAX** – returns the maximum value from a set(or multiset) of numeric values
- **MIN** - returns the minimum value from a set(or multiset) of numeric values
- **AVG** – returns the average of a set(or multiset) of numeric values

NOTE: Multiset is a collection similar to a set wherein the multiplicity of every value is greater than or equal to 1. In other words, it is a set that can contain duplicate values.

- The functions MAX and MIN can also be used with attributes that have non-numeric domains if the domain values have a total ordering among one another. DATE, TIME, TIMESTAMP, and alphanumeric strings are examples of such non-numeric domains.
- **Total order** - For any two values in the domain, it can be determined that one appears before the other in the defined order.

Example: Find the sum of salaries of all employees, the maximum salary, the minimum salary, and the average salary

Query: SELECT SUM(Salary), MAX(Salary), MIN(Salary), AVG(Salary) FROM EMPLOYEE;

Output:

SUM(Salary)	MAX(Salary)	MIN(Salary)	AVG(Salary)
281000.00	55000.00	25000.00	35125.000000

- The AS clause can be used to rename column names to more meaningful ones.

Query: SELECT SUM (Salary) AS Total_Sal, MAX (Salary) AS Highest_Sal, MIN (Salary) AS Lowest_Sal, AVG (Salary) AS Average_Sal FROM EMPLOYEE;

Output:

Total_Sal	Highest_Sal	Lowest_Sal	Average_Sal
281000.00	55000.00	25000.00	35125.000000

Example: Retrieve the total number of employees in the company.

Query: SELECT COUNT(*) FROM EMPLOYEE;

Output:

COUNT(*)
8

Example: Find the number of employees in the 'Research' department.

Query: SELECT COUNT(*) FROM EMPLOYEE, DEPARTMENT
WHERE DNO = DNUMBER AND DNAME = 'Research';

Output:

COUNT(*)
4

- In general, COUNT(*) can be used to find the number of rows present in the result of the query.

Example:

Find the sum of salaries of all employees from the 'Research' department, as well as the maximum salary, the minimum salary, and the average salary in this department.

Query: `SELECT SUM(Salary), MAX(Salary), MIN(Salary), AVG(Salary)`
`FROM (EMPLOYEE JOIN DEPARTMENT ON Dno = Dnumber)`
`WHERE Dname = 'Research';` //Tables are specified in FROM clause – so join operation

Output:

SUM(Salary)	MAX(Salary)	MIN(Salary)	AVG(Salary)
133000.00	40000.00	25000.00	33250.000000

- The COUNT function can be used to count the number of values in a column as well

Example: Count the number of distinct salary values in the database.

Query: SELECT COUNT(DISTINCT Salary) FROM EMPLOYEE;

Output:

COUNT(DISTINCT Salary)
6

- Note the usage of the DISTINCT in the previous query. To understand its significance, we shall run the same query without using DISTINCT.

Query: SELECT COUNT(Salary) FROM EMPLOYEE;

Output:

COUNT(Salary)
8

- SQL, by default, does not eliminate duplicates in the result of a query. Duplicate retention can be specified using the ALL keyword
- If duplicate elimination is required, it must be explicitly specified by using the DISTINCT keyword.
- Notice how there are three copies of the same value in the *Salary* column.
- The total number of values in the *Salary* column equals 8, and hence COUNT (*Salary*) returns 8 (no duplicate elimination).
- By using the DISTINCT keyword, we are specifying duplicate elimination. Hence, out of the three copies, two are discarded and one is retained. Therefore, COUNT(DISTINCT *Salary*) returns 6 (8 – 2 (discarded copies))

Salary
30000.00
40000.00
25000.00
38000.00
55000.00
43000.00
25000.00
25000.00

- In general, when an aggregate function is applied to a collection of values, NULL values are discarded before the calculation.
- **If the collection becomes empty because all the values are NULL, COUNT returns zero and other aggregate functions return NULL.**

Example: Find the average number of hours that employees work on each project.

Query: SELECT Pno, AVG(Hours) as avg_hours FROM WORKS_ON GROUP BY Pno;

Output:

Pno	avg_hours
1	26.25000
2	12.50000
3	25.00000
10	18.33333
20	12.50000
30	18.33333

- Let's have a look at *WORKS_ON* tuples that correspond to *Pno* = 20:

Essn	Pno	Hours
333445555	20	10.0
888665555	20	NULL
987654321	20	15.0

- Notice how one of the entries in the *Hours* column is NULL.
- When AVG is applied to the *Hours* column for *Pno* = 20, it discards the NULL value and considers the other two values. Therefore, the average computation for *Pno* = 20 is as follows:

$$(10 + 15) / 2 = 12.5$$

- Aggregate functions can also be used in selection conditions involving nested queries.
- They can be used in correlated nested queries, which are then placed in the WHERE clause of an outer query.

Example: Retrieve the names of all employees who have two or more dependents.

Query: SELECT Lname, Fname FROM EMPLOYEE

WHERE (SELECT COUNT(*) FROM DEPENDENT WHERE Ssn = Essn) >= 2;

Output:

Lname	Fname
Smith	John
Wong	Franklin

The correlated query returns the count of dependents for every employee, which is then checked if it is greater than or equal to 2. All tuples from the *EMPLOYEE* table that satisfy the said condition are then selected.

SQL has two functions that are used alongside comparison operators – **SOME** and **ALL**.

- The **SOME** function **returns True if at least one element in the collection is True.**
- The **ALL** function **returns True if all the elements in the collection are True.**

- In many cases, we would want to apply the aggregate functions to subgroups of tuples in a relation where the subgroups are based on some attribute values.
- For example, we may want to find the average salary of employees in each department or the number of employees who work on each project.
- In such cases, we need to partition the relation into non overlapping subsets(or groups)of tuples.
- The attributes used to partition a relation into groups are called **grouping attributes**. All tuples that have the same values for all grouping attributes are placed in the same group.
- **If NULLs exist in the grouping attribute, then a separate group is created for all tuples with a NULL value in the grouping attribute.**
- Aggregate functions can be applied independently to each group to produce summary information about each group.

- Grouping can be done in SQL using the GROUP BY clause.
- The GROUP BY clause specifies the grouping attributes. They should be present in the SELECT clause as well so that the value of the grouping attributes will appear in the result relation along with the aggregated values.

Example: For each department, retrieve the department number, the number of employees in the department, and their average salary.

Query: SELECT Dno, COUNT(*), AVG (Salary) FROM EMPLOYEE GROUP BY Dno;

Output:

Dno	COUNT(*)	AVG (Salary)
1	1	55000.000000
4	3	31000.000000
5	4	33250.000000

Query: SELECT Dno, COUNT(*), AVG (Salary) FROM EMPLOYEE GROUP BY Dno;

Output:

Dno	COUNT(*)	AVG (Salary)
1	1	55000.000000
4	3	31000.000000
5	4	33250.000000

- In the above query, *Dno* is the grouping attribute. The *EMPLOYEE* relation is partitioned into groups in such a way that each group has tuples with the same value of *Dno*.
- Hence, each group consists of employees who work for the same department.
- The COUNT and AVG functions are applied to each group independently and are displayed in the result relation.
- The SELECT clause must contain only the grouping attributes and aggregate functions applied on each group of tuples.

An illustration of how grouping works is provided for the previous example

Fname	Minit	Lname	<u>Ssn</u>	...	Salary	Super_ssn	Dno
John	B	Smith	123456789	...	30000	333445555	5
Franklin	T	Wong	333445555		40000	888665555	5
Ramesh	K	Narayan	666884444		38000	333445555	5
Joyce	A	English	453453453		25000	333445555	5
Alicia	J	Zelaya	999887777		25000	987654321	4
Jennifer	S	Wallace	987654321		43000	888665555	4
Ahmad	V	Jabbar	987987987		25000	987654321	4
James	E	Bong	888665555		55000	NULL	1

Dno	Count (*)	Avg (Salary)
5	4	33250
4	3	31000
1	1	55000

Result of Q24

Grouping EMPLOYEE tuples by the value of Dno

Example: For each project, retrieve the project number, the project name, and the number of employees who work on that project.

Query: SELECT Pnumber, Pname, COUNT(*) FROM PROJECT, WORKS_ON
WHERE Pnumber = Pno GROUP BY Pnumber, Pname;

Output:

Pnumber	Pname	COUNT(*)
10	Computerization	3
30	Newbenefits	3
1	ProductX	2
2	ProductY	3
3	ProductZ	2
20	Reorganization	3

- The above query is an example of using GROUP BY along with JOIN.
- In such statements, relations are first joined using the condition specified in the WHERE clause. Grouping and aggregation are then performed on the result of this joining

- There are some scenarios wherein we would want to retrieve the values of aggregate functions for groups that satisfy certain conditions.
- SQL provides the HAVING clause, which can appear in conjunction with a GROUP BY clause, for this purpose.
- **The HAVING clause is used to specify conditions for groups of tuples, whereas the WHERE clause is used to specify conditions for individual tuples.**
- HAVING provides a condition on the summary information regarding the group of tuples associated with each value of the grouping attributes.
- **Only the groups that satisfy the condition are retrieved in the result of the query.**

Example:

For each project on which more than two employees work, retrieve the project number, the project name, and the number of employees who work on the project.

Query:

```
SELECT Pnumber, Pname, COUNT(*) FROM PROJECT, WORKS_ON  
WHERE Pnumber = Pno GROUP BY Pnumber, Pname HAVING COUNT(*) > 2;
```

Output:

Pnumber	Pname	COUNT(*)
10	Computerization	3
30	Newbenefits	3
2	ProductY	3
20	Reorganization	3

An illustration of how the HAVING clause works is provided for the previous example

Pname	Pnumber	...	Essn	Pno	Hours
ProductX	1		123456789	1	32.5
ProductX	1		453453453	1	20.0
ProductY	2		123456789	2	7.5
ProductY	2		453453453	2	20.0
ProductY	2		333445555	2	10.0
ProductZ	3		666884444	3	40.0
ProductZ	3		333445555	3	10.0
Computerization	10	...	333445555	10	10.0
Computerization	10		999887777	10	10.0
Computerization	10		987987987	10	35.0
Reorganization	20		333445555	20	10.0
Reorganization	20		987654321	20	15.0
Reorganization	20		888665555	20	NULL
Newbenefits	30		987987987	30	5.0
Newbenefits	30		987654321	30	20.0
Newbenefits	30		999887777	30	30.0

These groups are not selected by the HAVING condition of Q26.

After applying the WHERE clause but before applying HAVING

Pname	Pnumber	...	Essn	Pno	Hours
ProductY	2		123456789	2	7.5
ProductY	2		453453453	2	20.0
ProductY	2		333445555	2	10.0
Computerization	10		333445555	10	10.0
Computerization	10	...	999887777	10	10.0
Computerization	10		987987987	10	35.0
Reorganization	20		333445555	20	10.0
Reorganization	20		987654321	20	15.0
Reorganization	20		888665555	20	NULL
Newbenefits	30		987987987	30	5.0
Newbenefits	30		987654321	30	20.0
Newbenefits	30		999887777	30	30.0

Pname	Count (*)
ProductY	3
Computerization	3
Reorganization	3
Newbenefits	3

Result of Q26
(Pnumber not shown)

After applying the HAVING clause condition

Example: For each project, retrieve the project number, the project name, and the number of employees from department 5 who work on that project.

Query: SELECT Pnumber, Pname, COUNT (*)
FROM PROJECT, WORKS_ON, EMPLOYEE
WHERE Pnumber = Pno AND Dno = 5 AND Ssn=Essn
GROUP BY Pnumber, Pname;

Output:

Pnumber	Pname	COUNT(*)
1	ProductX	2
2	ProductY	3
3	ProductZ	2
10	Computerization	1
20	Reorganization	1

- We must be extra careful when two different conditions apply – one specified in the WHERE clause and the other specified in the HAVING clause.
- **The rule is that the condition specified in the WHERE clause is executed first, to select the individual tuples, and then the HAVING clause is executed, to select groups of tuples.**
- **This means that the HAVING clause groups only those tuples that are selected by the WHERE clause.**
- Consider the following example:

Retrieve the count of number of employees whose salary exceeds \$40,000 in each department, but only for those departments where more than two employees work.

Query: SELECT Dno, COUNT(*) FROM EMPLOYEE

WHERE Salary>40000 GROUP BY Dno HAVING COUNT(*) > 2;

- At the outlook, this might seem like the correct query for the given question. However, closer inspection reveals that this might not be the case.
- As stated earlier, the WHERE clause is executed first and its result is what the HAVING clause works on.
- In other words, the WHERE clause first returns only those employees whose salary exceeds \$40,000. This is followed by the grouping of the filtered relation and the selection of groups by the HAVING clause condition.
- Hence, only departments with more than two employees who each earn more than \$40,000 are selected, which is not the expected result.

The correct query is given below

Query: SELECT Dno, COUNT(*) FROM EMPLOYEE
WHERE Salary>40000 AND Dno IN
(SELECT Dno FROM EMPLOYEE GROUP BY Dno HAVING COUNT(*) > 2)
GROUP BY Dno;

Output:

Dno	COUNT(*)
4	1

This shows that there is only one employee with a salary greater than \$40,000 belonging to a department(here department 4) with more than two employees. All other departments either have two or fewer employees or don't have employees whose salary exceed \$40,000.

Let's try executing the incorrect query

Query: SELECT Dno, COUNT(*) FROM EMPLOYEE

WHERE Salary>40000 GROUP BY Dno HAVING COUNT(*) > 2;

Output:

Dno	COUNT(*)
-----	----------

- The output of the query is an empty relation. This is because the query returns only those departments with more than two employees whose salaries exceed \$40,000.
- From the output of the correct query, we can see that there are no departments that satisfy this condition. Hence, an empty relation is returned.

Aspect	WHERE	HAVING
Purpose	Filters rows before grouping/aggregation	Filters groups/aggregated results after GROUP BY
When is it Applied	Applied before grouping	Applied after grouping
Works On	Works on individual row values (columns)	Works on aggregated values (like sum(), count() etc)
Use of Aggregates	Cannot use aggregate functions.	Can use aggregate functions.
Order of execution	First WHERE is executed then subsequent statements	HAVING is executed after WHERE has been applied

1) Which of the following built-in aggregate functions can be used with non-numeric domains (such as DATE, TIME, and alphanumeric strings) provided there is a total ordering among the values?

- A) SUM and AVG
- B) COUNT and SUM
- C) MAX and MIN
- D) AVG and MIN

2) If a collection of values becomes empty because all the values are NULL, the COUNT function returns zero, but all other aggregate functions return _____.

3) Fill in the blank: While the WHERE clause is used to specify conditions for individual tuples, the HAVING clause is exclusively used to specify conditions for _____ of tuples.

4) When using the GROUP BY clause, what happens if NULL values exist within the specified grouping attribute?

- A) The query throws a syntax error.
- B) The tuples with NULL values are completely discarded from the result.
- C) A separate group is created for all tuples with a NULL value in the grouping attribute.
- D) The NULL values are randomly assigned to other existing groups.

5) When a single SQL query contains both a WHERE clause and a HAVING clause, what is the correct order of execution?

- A) The HAVING clause is executed first to filter groups, then WHERE filters the individual rows.
- B) The WHERE clause is executed first to select individual tuples, and then the HAVING clause is executed to select groups of tuples.
- C) They are executed simultaneously.
- D) The execution order depends on the aggregate functions used in the SELECT clause.

6) When using functions alongside comparison operators in nested queries, which function returns True if all the elements in the collection evaluate to True?

- A) SOME
- B) EXISTS
- C) ANY
- D) ALL

Answers:

1. C
2. NULL
3. groups (or subgroups)
4. C
5. B
6. D



THANK YOU

Dr. Nagasundari S

Department of Computer Science and Engineering

nagasundaris@pes.edu